FOR THE DEVELOPMENT TEAM

Map King Command Center

A read-and-decide layer over the systems Map King already runs on. It aggregates, ranks and launches. It does not store work.

THE ONE RULE THAT GOVERNS EVERY SCREEN

Attio owns customers and deals. Linear owns tickets. Airtable owns agent output and approvals. This app reads them, ranks what matters, and hands the CEO back to the source. Every task-like object carries a source badge and an "Open in [source]" action. No screen may imply a record is created and stored here.

SCOPE AND AUDIENCE

Single user in v1: the CEO. Other team members and vendor firms are read-only stakeholders whose names appear as owners and assignees. Browser app, desktop-first, usable down to tablet. The clickable prototype is the visual and behavioral reference; this document is the functional breakdown behind it.

Navigation

Nine sidebar destinations. Grouped destinations carry a tab row at the top of the page; tab state persists per group for the session.

DESTINATION	TABS	PRIMARY SOURCE
Dashboard	—	Derived from all
Today	—	Derived from all
Calendar	—	Google Calendar
Work	Queue · Delivery	All · Airtable
Revenue	Pipeline · Subscriptions · Ads · Accounts	Attio · Stripe
Product	Tickets · Features · Updates · Agents · QA feed · QA reports	Linear
Brain	MKOS files · Library · Sessions · Audits	MKOS store
Company	Telemetry · Team & vendors	Stripe · QuickBooks · platform
Approvals	—	Airtable

How to read section 3

One page per surface. Each gives the purpose, the behavior to build, the data it depends on, and the questions that need an answer before that surface can be estimated. The questions are the part worth arguing with.

Dashboard

The landing page. An agent-written brief plus seven tiles that surface the single most important state from each other page.

BEHAVIOR

- Brief is three to four sentences, each with evidence chips naming the screen and figure behind the claim.
- A "Since you last opened it" list diffs against the CEO's last session timestamp.
- One recommended action closes the brief, with two buttons that route to the relevant screens.
- Two large tiles (top fire, Needs you) and five small ones. Tile header routes to the owning page; a More control expands detail in place.

DATA

Derived only. The dashboard holds no records of its own — every tile reads the same store as its owning page.

OPEN QUESTIONS

- Is the brief generated on page load, on a schedule, or cached until the underlying data changes? Load-time generation costs latency on the most-visited page.
- What defines "last opened" — a server-side timestamp per user, or local storage?
- If a tile has nothing to report (zero approvals, no critical QA), does it hide, show an empty state, or get replaced by the next-ranked tile?

Today

The daily operating screen: at most three fires, the day's blocks, a power-hour checklist, and the people the CEO is blocking.

BEHAVIOR

- Hard cap of three fires. Adding a fourth opens a modal that requires demoting one; the demoted fire drops to the This Week lane, it is not deleted.
- Each fire shows a literal next action, owner, due date, days-in-state, domain tag and source badge, with Start and Open-in-source actions. Start arms a focus block.
- Needs You lists items blocking other people, sorted by days waiting, red past three days.
- Power hour is a checklist of sub-30-minute items with source badges; completion is local to the session unless written back.

DATA

Fires are a ranked selection over the aggregated task set, not a separate object type. Ranking and rank order must persist per user.

OPEN QUESTIONS

- Where does fire rank persist, and is it user-ordered or recomputed? The prototype treats it as user-ordered.
- Does checking a power-hour item write completion back to its source system, or only mark it locally?
- Does "Start a focus block" create a real calendar event, or is it a UI state only?
- Who computes "days in state" — us from a status-change history, or does each source expose it?

Calendar

Week grid where the CEO drags time blocks around fixed calls, choosing what to work on by category, tag and stakeholder.

BEHAVIOR

- Five-day grid, 7am to 7pm, thirty-minute resolution. Drag vertically to create a block.
- Left rail sets the block category (Deep work, Power hour, Admin, Buffer), plus tag and stakeholder filters that narrow the list of work items.
- Calls and meetings render as fixed, non-editable blocks. A user block overlapping one gets a conflict border and increments a conflict count.
- Blocks are removable. Mouseup anywhere on the window commits the drag.

DATA

Fixed events read from Google Calendar. User blocks are the one genuinely app-owned object in the product.

OPEN QUESTIONS

- Do user blocks write back to Google Calendar as real events, or live only in our store? This is the largest single scope decision in the app.
- Should existing blocks be draggable and resizable after creation, or is delete-and-redraw acceptable for v1?
- Is the week fixed Monday to Friday, or does it need weekends and week navigation?
- Multiple calendars, or one? Conflict detection depends on which are in scope.

Work · Queue

Every task from every source in one filterable list, organized into lanes for triage.

BEHAVIOR

- Filters by domain, owner and source; lanes for On Fire, This Week, Waiting On, Queued and Icebox, each showing a live count.
- Anything past its date with no update gets a stale badge and a forced decision control: Done, Drop or Reschedule, inline.
- Tasks owned by the CEO that could be delegated show a "Delegate to [person]?" prompt.
- Multi-select enables bulk retag, reassign and push-to-next-week.

DATA

A normalized task record across Attio, Linear, Airtable and manual entries.
Needs a stable cross-source identity and a last-updated timestamp.

OPEN QUESTIONS

- What exactly makes a task stale — days since last update, or days past due with no status change? These give different lists.
- Do Done, Drop and Reschedule write to the source system? Reschedule in particular needs a date field that not all sources have.
- What rule drives the delegate prompt? The prototype uses a hand-set flag; a real rule needs a definition.
- How are manual tasks stored, given the no-system-of-record principle? A thin manual store is the pragmatic answer but breaks the rule.

Work · Delivery

Board of every event build in flight, grouped by the six pipeline stages, with a capacity meter above it.

BEHAVIOR

- Stages: Content Ingestion, Auto-Build, QA, Launch Kit, Live, Post-Event Intelligence.
- Cards show customer, event date, days to launch, percent complete, owner, open QA issues by severity, and content volume. A card turns red when days-to-launch falls inside the build-time estimate for its event type.
- The capacity meter shows committed against available hours, with proposed-unsigned hours as a separate segment.
- Clicking a card opens a drawer with that build's runbook checklist and step completion.

DATA

Builds and runbook steps from Airtable. Build-time estimates per event type must live somewhere editable.

OPEN QUESTIONS

- Where do the per-event-type build estimates live — a config, or a field on the account?
- Is percent complete a count of completed runbook steps, or a separate field someone maintains?
- Are runbook steps tickable from this drawer, and does that write to Airtable?
- How are delivery hours captured — estimated per build, or tracked against actuals?

Revenue

Four tabs: a revenue dashboard splitting subscription from ad revenue, the deal pipeline, recurring programs, sponsor inventory, and per-account detail.

BEHAVIOR

- Pipeline is a kanban by stage. Days-in-stage is the hero metric: cards fade, then flag red as they age. A "deals with no next step" count sits in red at the top.
- Every deal card has a Price it action opening a calculator: placemarks, content pages, attendance and event type produce prices at all three tiers, with overage breakdown, a portfolio-rate toggle and a recommended tier.
- Subscriptions lists recurring programs with tier, renewal date and status. Ads lists sponsor inventory per event with sell-through, impressions and tap-through.
- Accounts opens per-customer detail: metrics, call log, open tasks, deals, platform data, contracts, contacts, and a notes-and-ideas thread the CEO can add to.

DATA

Deals and accounts from Attio; subscription and invoice data from Stripe; sponsor performance from the platform. Pricing rates must be configurable, not hard-coded.

OPEN QUESTIONS

- Where do tier base prices and overage rates live? They change, and the calculator, the proposal agent and the MKOS Sales Playbook all need the same numbers.
- Is the recommended tier always the cheapest that fits, or is there a margin rule?
- What does "Generate proposal draft" do — call the proposal agent and land a draft in Approvals, or open a template in the Library?
- Weighted forecast uses fixed per-stage weights in the prototype. Should those be derived from historical close rates?
- Are account notes and ideas stored in Attio or in our own store?

Product

Migration tracker, Linear tickets filtered to CEO decisions, a feature request list, an update log, the agent build board, the QA feed, and full QA reports.

BEHAVIOR

- Migration tracker shows percent complete, blockers and who is waiting on each.
- Tickets are grouped by epic and default to the "needs CEO decision" filter.
- Features and Updates each have a composer that adds an entry to the top of the list.
- Agent board shows each pipeline agent with Manual / In Build / Live state, an automation percentage and its vendor owner.
- QA reports list each agent run with severity counts. Inside a report, each finding can be accepted — creating a Linear ticket filed under that report — or dismissed. Accept-all triages the remainder.

DATA

Tickets and epics from Linear; QA runs and findings from the QA agent.
Features and updates need an app-owned store.

OPEN QUESTIONS

- How is "needs CEO decision" expressed in Linear — a label, a custom field, or an assignee convention?
- When a finding is accepted, what does the created ticket contain, and how does the report-to-ticket link survive in Linear? A label per report is the obvious mechanism.
- Should accepting a finding be reversible after the ticket exists?
- Where do feature requests live long-term, and what promotes one into a Linear ticket?
- Automation percentage per agent — measured, or hand-maintained?

Brain · MKOS files

The six context files every session, agent and proposal reads, with visibility, comments and AI-suggested updates.

BEHAVIOR

- Index shows each file with purpose, version, staleness and count of waiting suggestions.
- File detail shows the contents by section, a notes-and-prompts thread the CEO can add to, and suggested updates.
- Each suggestion carries a rationale, the command-center evidence behind it, and a line-level diff. Approve promotes it to a new version; dismiss discards it. Nothing writes without the approval.

DATA

App-owned versioned document store. Versions, authorship and diff history must be retained.

OPEN QUESTIONS

- What format are the files — markdown with headings, or structured sections? The diff view assumes line-level text.
- Is there rollback to a previous version, and is version history browsable?
- What triggers suggestion generation — a schedule, a data change, or the CEO asking?
- Do other team members or the vendor firms ever read these files, and if so do they need a permission model?

Brain · Library, Sessions, Audits

Templates and prompts, the strategy-session chat, and the four-phase AI audit loop.

BEHAVIOR

- Library holds internal and external templates plus MKOS prompts, searchable and filterable, each with a copy action and an agent reshape panel that produces a draft variant without overwriting the saved template.
- Sessions loads a domain's context into a chat, and outputs proposed context-file changes as a diff with Promote and Discard.
- Audits run four phases: Analysis (findings with evidence), Questions (what the data cannot tell the agent, answered by tap), Decisions (options with a marked recommendation), Changes (each resulting change applied to the system that owns it, individually or all at once).

DATA

Templates and prompts in an app-owned store. Audit state — answers, decisions, applied actions — must persist per audit run.

OPEN QUESTIONS

- Are audits scheduled, triggered manually, or both? And is an audit run immutable once created, or does it re-analyze when the data moves?
- When an audit change says "apply to Linear" or "apply to Queue", what is the actual write? Each of the eighteen prototype actions needs a concrete implementation or needs downgrading to a prompt.
- Do template fill-in variables resolve from a real account record, or is that a copy-and-edit flow?
- Are session transcripts retained, and for how long?

Company

Telemetry the CEO checks rather than watches, plus the team, vendor and platform roster.

BEHAVIOR

- Big quiet numbers with sparklines and month-over-month deltas: booked, collected and recurring revenue, ad revenue, events delivered and in flight, content pieces built, total users and adoption, data rows, delivery hours against capacity, automation percentage.
- One hero chart: revenue against delivery capacity over twelve months.
- Team and vendor cards show committed-against-capacity load, what each owns and what each is blocked on. A platforms list shows every integration with connection state, last sync and which screens it feeds.

DATA

Stripe and QuickBooks for money; the platform for usage; the delivery store for hours.

OPEN QUESTIONS

- What is the source of truth for delivery hours consumed — estimates, or tracked time? Everything capacity-related depends on this answer.
- Are vendor hours self-reported, invoiced, or tracked?
- Is "shipped this week" per person compiled automatically from Linear and Airtable activity, or written by hand?
- Do metrics need a date-range control, or is YTD-plus-MTD enough?

Approvals

The human gate. Every agent-generated artifact that would reach a customer waits here.

BEHAVIOR

- Pending list of proposals, follow-ups, invoices and reports, each with type, account and generating agent.
- Selecting an item shows its full content in a preview pane.
- Approve, Edit, or Reject with a required reason. Approve queues the send from the owning system; reject returns it to the agent with the reason attached.
- A standing banner states the rule: agents draft, nothing goes to a client until approved.

DATA

Drafts from Airtable. Approval state, actor and timestamp must be auditable.

OPEN QUESTIONS

- Does Approve actually send, or does it mark approved and let the source system send? These differ materially in failure modes.
- Does Edit open an inline editor here, or bounce to the source?
- Is the reject reason fed back into the agent as training context, or just stored?
- Does anything expire or escalate if it sits unapproved?

Integrations

Every integration is read-heavy. Writes are narrow and explicit, and each one is listed here — if a write is not on this list, the app does not perform it.

PLATFORM	READS	WRITES
Attio	Accounts, contacts, deals, stages, activity	Next step, stage change, logged note
Linear	Issues, epics, labels, state, assignee	Create issue from QA finding or audit action
Airtable	Builds, runbook steps, agent output, approval queue	Approval state, runbook step completion
Gmail	Client threads for the account call log	Send on approval (if Approve sends)
Google Calendar	Calls and meetings for Today and Calendar	Time blocks (pending the decision on page 16)
Stripe	Invoices, payments, subscription revenue	None
QuickBooks	Vendor spend, monthly close	None
Platform (Supabase / Bubble)	Users, adoption, content volume, data rows, sponsor impressions	None
Slack	Not connected in v1	None

AI surfaces

Five places call a model. All five are gated: the model proposes, the CEO promotes. Nothing an agent produces reaches a client or a stored file without an explicit human action.

Morning brief

Summarizes overnight state across every connected source into three or four sentences with evidence, a change list and one recommended action.

Gate: Read-only output. No writes.

MKOS suggestions

Proposes line-level edits to context files, justified by command-center evidence.

Gate: Approve per suggestion; promotes to a new file version.

Strategy sessions

Domain-scoped chat loaded with the relevant context file, ending in a proposed diff.

Gate: Promote or Discard. Nothing writes without Promote.

Audits

Cross-platform analysis, clarifying questions, decision framing, then applied changes.

Gate: The CEO answers, decides, and applies each change.

Drafting agents

Proposals, follow-ups, invoices and post-event reports generated from templates and account data.

Gate: Everything lands in Approvals. Nothing reaches a client unapproved.

Settle these before build

Each of these changes scope materially. None of them can be deferred past phase 1 planning.

Does this app write to Google Calendar?

Time blocking is the one feature that either owns real calendar events or is a private overlay. It changes the auth scope, the sync model and the conflict logic.

Is there any app-owned task store at all?

The read-and-decide principle says no, but manual tasks, fire ranking, power-hour completion and lane assignment all need somewhere to live. Recommend a thin state layer that stores only ranking and app-local flags, never task content.

Sync model: polling, webhooks, or on-demand?

The dashboard and Today read from every source at once. Webhooks where available, with a cached read layer, is the likely answer, but it determines how fresh "synced 4 min ago" really is.

Where does pricing configuration live?

Tier prices, overage rates, event-type multipliers and the portfolio discount are used by the calculator, the proposal agent and the Sales Playbook. One source, editable without a deploy.

Which model, and what is the context budget?

The brief, audits and sessions each load substantial cross-platform context. This drives cost per user per day and the caching strategy.

Single user or multi-user in v1?

Other team members appear throughout as owners. If they log in, permissions, per-user state and the MKOS comment thread all change shape.

What happens when a source is down or unauthorized?

Every screen is an aggregate. Per-source degradation states need designing rather than a page-level error.

Audit action implementations

The eighteen audit actions in the prototype range from a one-line MKOS write to reassigning tasks across systems. Each needs to be classified as automated, semi-automated, or a prompt completed manually.

Suggested build order

Ordered so that each phase is usable on its own. Phase 1 alone replaces the daily habit the app exists to fix.

1

Read layer and the daily loop

Attio, Linear, Airtable and Calendar connectors; normalized task model; Today, Work and the dashboard shell without the AI brief. This alone replaces the morning scramble.

2

Revenue and delivery depth

Pipeline with days-in-stage, the pricing calculator, Accounts detail, the delivery board and capacity meter, Approvals with its gate.

3

AI surfaces

Morning brief, MKOS file store with suggestions and promotion, strategy sessions, the Library. Gated writes throughout.

4

Audits and calendar writes

The four-phase audit loop with concrete applied actions, and time blocking with write-back if that decision lands in favor.